

homework 6

1.

1. What is the hash table? What is the hash function? Is there a first and last element in the hash table?

מהי טבלת גיבוב (Hash Table)? טבלת גיבוב היא מבנה נתונים המשמש לאחסון ושליפה מהירה מאוד של נתונים בזוגות של מפתח וערך (Key-Value), לרוב בסיבוכיות זמן ממוצעת של $O(1)$.

מהי פונקציית גיבוב (Hash Function)? פונקציה המקבלת מפתח (Key) וממירה אותו לאינדקס מספרי. האינדקס הזה מצביע על המיקום המדויק (בתא במערך) שבו יישמר הערך המשויך לאותו מפתח.

האם יש איבר ראשון ואחרון בטבלת גיבוב? במבנה הסטנדרטי – לא. טבלת גיבוב אינה שומרת על סדר ההכנסה של הנתונים ואינה ממיינת אותם. המיקום של כל איבר נקבע באופן אקראי על ידי פונקציית הגיבוב, ולכן אין משמעות לאיבר "ראשון" או "אחרון".

2.

2. What is the purpose of the hash function? Are there any functions that are better than others? How is this measured? Does the function have to be single-valued? What happens if it doesn't?

מהי מטרת פונקציית הגיבוב? מטרתה לקחת קלט של מפתח (Key) ולהמיר אותו לאינדקס מספרי בתוך גבולות המערך של טבלת הגיבוב. זה מאפשר גישה ישירה, שמירה ושליפה מהירה מאוד של נתונים בזמן ממוצע של $O(1)$.

האם יש פונקציות טובות מאחרות וכיצד זה נמדד? כן. פונקציית גיבוב "טובה" נמדדת בשני מדדים עיקריים:

1. פיזור אחיד: היכולת של הפונקציה לפזר את המפתחות השונים באופן שווה על פני כל תאי הטבלה, במטרה למזער ככל הניתן "התנגשויות" (מקרים שבהם שני מפתחות שונים מקבלים את אותו אינדקס).
2. יעילות חישוב: הפונקציה צריכה להיות מהירה ופשוטה לחישוב כדי לא לעכב את תהליך ההכנסה והשליפה.

האם הפונקציה חייבת להיות חד-ערכית (דטרמיניסטית)? כן, בהחלט. פונקציית גיבוב חייבת להחזיר תמיד את אותו ערך מחושב עבור אותו מפתח בדיוק.

מה קורה אם היא אינה חד-ערכית? אם הפונקציה תחזיר ערכים שונים עבור אותו מפתח בזמנים שונים, המערכת תקרוס לוגית. אנחנו נשמור נתון בתא מסוים לפי אינדקס מסוים, אך כשננסה לשלוף אותו מאוחר יותר עם אותו מפתח, הפונקציה תכוון אותנו לאינדקס אחר לגמרי – והנתון פשוט "יאבד" בתוך הטבלה ולא נצליח למצוא אותו.

3.

3. What is the advantage of the search function? What is the efficiency of searching in a list, and what is the efficiency of searching in a set? What is the difference?

מהו היתרון של פונקציית החיפוש (במבני נתונים מבוססי גיבוב)? היתרון המרכזי הוא מהירות עצומה. חיפוש בטבלת גיבוב מאפשר למצוא נתון בזמן קבוע (בממוצע), ללא קשר לכמות הנתונים השמורה במבנה.

מהי יעילות החיפוש ברשימה (List)? יעילות החיפוש ברשימה רגילה היא ליניארית, כלומר $O(n)$. המשמעות היא שכדי למצוא איבר מסוים, האלגוריתם נדרש לסרוק את איברי הרשימה בזה אחר זה עד למציאת הפריט (או עד סוף הרשימה). ככל שהרשימה ארוכה יותר, פעולת החיפוש תיקח יותר זמן.

מהי יעילות החיפוש בקבוצה (Set)? יעילות החיפוש בקבוצה (Set) היא בממוצע $O(1)$ (זמן קבוע). קבוצות בשפות תכנות רבות (כמו פייתון) ממומשות מאחורי הקלעים באמצעות טבלת גיבוב, ולכן פעולת החיפוש בהן מיידית.

מהו ההבדל? ההבדל נובע מאופן אחסון הנתונים: רשימה מסדרת את הנתונים באופן עוקב (סידורי), ולכן מחייבת סריקה איבר-איבר. קבוצה (Set) לעומת זאת, משתמשת בפונקציית גיבוב כדי לחשב בדיוק היכן בזיכרון נמצא כל איבר. כשאנחנו מחפשים נתון ב-Set, המערכת מחשבת את הגיבוב שלו ו"קופצת" ישירות למיקום המתאים בזיכרון, מבלי לעבור על שאר האיברים. לכן, עבור פעולות של חיפוש ובדיקת שייכות, Set יעיל פי כמה מרשימה.

4.

4. Can every variable be on the hash table? What is the concern in the case of a variable that is not hashable? Which types are hashable and which are not?

האם כל משתנה יכול להיות בטבלת גיבוב? לא, לא כל משתנה יכול לשמש כמפתח (Key) בטבלת גיבוב. כאיבר ב-Set. רק משתנים המוגדרים כ"ברי-גיבוב" (Hashable) יכולים לשמש למטרה זו. עם זאת, חשוב לציין שכערך (Value) המוצמד למפתח, ניתן לשמור כל סוג של משתנה ללא הגבלה.

מה החשש במקרה של משתנה שאינו ברי-גיבוב? הבעיה נעוצה במשתנים שתוכנם יכול להשתנות לאחר יצירתם (Mutable). פונקציית הגיבוב מחשבת את מיקום הנתון בטבלה על בסיס הערך של המפתח. אם נשתמש במשתנה שניתן לשינוי כמפתח, ונשנה את התוכן שלו לאחר שהוא כבר הוכנס לטבלה, ערך הגיבוב שלו ישתנה גם כן. התוצאה היא "שבירה" של הטבלה: כאשר ננסה לחפש את המפתח, הפונקציה תפנה אותנו לאינדקס חדש ושגוי, והנתון שהכנסנו קודם לכן פשוט ילך לאיבוד במערכת ולא נוכל לגשת אליו.

אילו סוגי נתונים הם ברי-גיבוב ואילו לא? ככלל אצבע, סוגי נתונים קבועים שאינם ניתנים לשינוי (Immutable) הם ברי-גיבוב, וסוגים הניתנים לשינוי (Mutable) אינם ברי-גיבוב.

- סוגים ברי-גיבוב (Hashable): * מספרים (שלמים - int, ועשרוניים - float)
- מחרוזות טקסט (string)
- ערכים בוליאנים (boolean)

- טאפלים (Tuple) - בתנאי שכל האיברים בתוכם הם גם ברי-גיבוב.
- סוגים שאינם ברי-גיבוב (Not Hashable):
- רשימות (list)
- מילונים (dictionary / dict)
- קבוצות (set)

5.

5. Is there order in a set? Is there order in a dictionary (present and past)? How is this possible?

האם יש סדר בקבוצה (Set)? לא. קבוצות (Sets) מבוססות על טבלת גיבוב בסיסית ואינן שומרות על סדר ההכנסה של האיברים. אם נדפיס את איברי הקבוצה או נעבור עליהם בלולאה, הסדר שבו הם יופיעו ייקבע על ידי ערכי הגיבוב שלהם, ולמשתמש הוא ייראה אקראי לחלוטין.

האם יש סדר במילון (Dictionary) כיום ובעבר? כאן יש הבדל משמעותי התלוי בגרסאות (התשובה מתייחסת ספציפית לשפת Python, שהיא השפה הנפוצה ביותר בהקשר זה):

- **בעבר (עד גרסת פייתון 3.5 כולל):** לא היה סדר. מילונים התבססו על טבלת גיבוב קלאסית ולא הבטיחו שמירה על סדר ההכנסה.
- **כיום (החל מגרסת פייתון 3.7 באופן רשמי, וב-3.6 כמאפיין יישום):** כן. מילונים כיום שומרים על סדר ההכנסה של הנתונים (Insertion Order). כאשר נעבור על המילון, הזוגות (מפתח-ערך) יופיעו בדיוק בסדר שבו הוספנו אותם.

כיצד זה מתאפשר (הרי טבלת גיבוב לא שומרת על סדר)? הדבר מתאפשר בזכות שינוי חכם בארכיטקטורה הפנימית של המילון בפייתון מודרני. במקום להשתמש בטבלת גיבוב אחת גדולה שבה מפוזרים הנתונים, המילון החדש מורכב משני חלקים:

1. **מערך "צפוף" (Dense array):** כאן נשמרים המפתחות והערכים עצמם, והם פשוט נדחפים למערך זה בזה אחר זה, בדיוק לפי סדר ההכנסה. זה מה שמאפשר למילון לזכור את הסדר.
2. **טבלת גיבוב "דלילה" (Sparse array):** טבלה זו אינה שומרת את הנתונים עצמם, אלא משמשת רק כאינדקס (מפתח עניינים). היא מקבלת את המפתח, מחשבת לו פונקציית גיבוב, ושומרת רק את המיקום (האינדקס) שלו בתוך המערך הצפוף.

בזכות ההפרדה הזו, אנו נהנים מכל העולמות: כשאנחנו מבצעים פעולת חיפוש, פונקציית הגיבוב מוצאת מיד את האינדקס בטבלה הדלילה (מהירות של $O(1)$). מנגד, כשאנחנו עוברים על המילון בלולאה, הפייתון פשוט עובר על המערך הצפוף מההתחלה ועד הסוף, וכך סדר ההכנסה נשמר בצורה מושלמת.

6.

6. The * operator has many uses, what are they? (with an emphasis on mathematical operations, operation parameters, and unpacking)

1. פעולות מתמטיות ושכפול (Mathematical Operations):

- כפל בסיסי: השימוש הפשוט ביותר הוא ביצוע פעולת כפל רגילה בין שני מספרים (לדוגמה: ביטוי כמו $3 * 4$ יחזיר 12).
- שכפול רצפים (Repetition): כאשר משתמשים באופרטור הכוכבית בין רצף (כמו מחרוזת טקסט, רשימה או טאפל) לבין מספר שלם (Integer), הפעולה תשכפל את הרצף כמספר הפעמים שצוין. לדוגמה: הכפלת המחרוזת "Hi" ב-3 תייצר מחרוזת חדשה: "HiHiHi". הכפלת הרשימה [1, 2] ב-2 תייצר את הרשימה [1, 2, 2, 1].
- חזקה (כוכבית כפולה): שימוש בשתי כוכביות רצופות משמש לפעולת חזקה מתמטית (לדוגמה: 2^3 משמעותו 2 בחזקת 3, והתוצאה תהיה 8).

2. פרמטרים בפונקציות (Operation Parameters - `_args`): כאשר מגדירים פונקציה, ניתן להוסיף כוכבית לפני שם של פרמטר (המוסכמה הנפוצה היא לקרוא לו `_args`). פעולה זו מאפשרת לפונקציה לקבל כמות גמישה ובלתי מוגבלת של ארגומנטים (Positional arguments). מאחורי הקלעים, פייתון אוספת את כל הערכים הבודדים שהמשתמש הזין ו"אורזת" אותם יחד אל תוך טאפל (Tuple) אחד שעליו הפונקציה יכולה לבצע פעולות, כמו מעבר בלולאה. (כדאי להזכיר גם את `kwargs` - שתי כוכביות המאפשרות לפונקציה לקבל כמות לא מוגבלת של ארגומנטים בעלי שם (Keyword arguments), שאותם פייתון אורזת אל תוך מילון).

3. פריקת איברים (Unpacking):

זהו למעשה התהליך ההפוך מסעיף 2, והוא משמש לפירוק קולקציות (כמו רשימות או טאפלים) לאיברים בודדים:

- פריקה בקריאה לפונקציה: אם פונקציה מסוימת מצפה לקבל שלושה משתנים נפרדים, ויש לנו רשימה עם שלושה איברים, הוספת כוכבית לפני שם הרשימה בזמן הקריאה לפונקציה "תפרק" אותה כך שכל איבר ברשימה יישלח כמשתנה נפרד.
- פריקה בהשמת משתנים (Assignment): מאפשר לפצל רשימה למספר משתנים. לדוגמה, אם יש לנו רשימה של חמישה איברים ונרצה לשמור את האיבר הראשון במשתנה נפרד ואת כל השאר במשתנה אחר, נוכל לכתוב: `a, *b = [1, 2, 3, 4, 5]`. התוצאה: המשתנה `a` יקבל את המספר 1, והמשתנה `b` יקבל רשימה של שאר האיברים [2, 3, 4, 5].
- מיזוג רשימות: דרך נוחה מאוד לאחד מספר רשימות או קבוצות לרשימה אחת חדשה. למשל, הגדרת רשימה המורכבת מפריקה של רשימות קודמות: `[list1, list2]`.

7.

7. Is a range a type of container? What's the difference between them? When should you use each?

מבחינה טכנית, בפייתון `range` נחשב לסוג של רצף (Sequence), אך הוא פועל בצורה שונה מקונטיינרים

קלאסיים (כמו רשימה, מילון או קבוצה). בניגוד לקונטיינר רגיל, range אינו מאחסן או "מכיל" פיזית את כל האיברים שלו בזיכרון בזמן-הריצה.

מה ההבדל ביניהם? ההבדל המרכזי הוא בצריכת הזיכרון ובאופן החישוב: קונטיינר רגיל (כמו רשימה) מקצה מקום בזיכרון ושומר בתוכו את כל האיברים. אם ניצור רשימה של מיליון מספרים, המחשב יצטרך לשמור מיליון ערכים בזיכרון. לעומת זאת, range פועל בשיטה "עצלנה" (Lazy evaluation). הוא שומר בזיכרון אך ורק שלושה נתונים: מספר התחלה (start), מספר סיום (stop) וגודל הקפיצה (step). את המספרים עצמם הוא מחשב תוך כדי תנועה (On the fly) רק מתי שמבקשים אותם (למשל כשעוברים עליו בלולאה). לכן, range של 10 מספרים ו-range של מיליארד מספרים יצרכו בדיוק את אותה כמות קטנה של זיכרון.

מתי כדאי להשתמש בכל אחד?

- **שימוש ב-range:** מומלץ מאוד כאשר צריך לייצר סדרת מספרים שלמים עוקבים לצורך חזרתיות, למשל כדי להריץ לולאת for מספר מוגדר של פעמים. זהו הפתרון היעיל והחסכוני ביותר בזיכרון לטווחים גדולים של מספרים.
- **שימוש בקונטיינר (כגון רשימה):** מומלץ כאשר הנתונים שצריך לאחסן אינם מסודרים בסדרה חשבונית פשוטה, כאשר מדובר בסוגי נתונים שאינם מספרים (טקסטים, אובייקטים וכו'), כאשר צריך לעדכן ולשנות את הנתונים לאחר יצירתם (להוסיף/למחוק איברים), או כאשר צריכים לגשת לאיברים בסדר אקראי שוב ושוב.

תרגיל:

Give an example of a possible hash function, for a class of students if we want to store information about them in a computerized form.

דוגמה לפונקציית גיבוב (Hash Function) אפשרית לשמירת מידע על תלמידים בכיתה:

בחירת המפתח (Key): הצעד הראשון הוא לבחור מפתח שיהיה ייחודי לכל תלמיד. בחירה מתבקשת וטובה תהיה **מספר תעודת הזהות (ת"ז)** או מספר הסטודנט שלו. מזהים אלו הם ייחודיים לחלוטין, מה שמבטיח שכל תלמיד יקבל התייחסות נפרדת (שימוש בשם התלמיד כמפתח הוא רעיון רע, שכן יכולים להיות שני תלמידים בעלי אותו שם, מה שייצור התנגשות ודאית).

הפונקציה המוצעת - חלוקה בשארית (Modulo): נניח שהקצינו בזיכרון טבלת גיבוב (מערך) בגודל N תאים כדי לאחסן את נתוני התלמידים. פונקציית הגיבוב שלנו תיראה כך: אינדקס = מספר תעודת הזהות מודולו N (או באנגלית: $\text{Index} = \text{ID} \% N$)

איך זה עובד בפועל ולמה זו בחירה טובה?

1. הפונקציה לוקחת את מספר תעודת הזהות של התלמיד (שהוא מספר בעל 9 ספרות).
2. היא מחלקת אותו בגודל המערך שלנו (N) ומחזירה רק את השארית של החלוקה.

3. מתמטית, שארית החלוקה ב-N תמיד תייצר מספר שלם בטווח שבין 0 לבין $N-1$. תכונה זו מבטיחה שהאינדקס שהפונקציה פולטת תמיד ייפול בדיוק בתוך הגבולות החוקיים של תאי המערך שלנו.
4. יעילות ופיזור: פעולת השארית (Modulo) במחשב היא מהירה מאוד לחישוב. בנוסף, אם נבחר את גודל הטבלה (N) בצורה חכמה (למשל, מספר ראשוני שרחוק מחזקות של 2), הפונקציה תפזר את מספרי תעודות הזהות השונים בצורה אקראית ואחידה יחסית על פני כל תאי הטבלה, מה שיקטין למינימום את הסיכוי ששני תלמידים יקבלו את אותו אינדקס (התנגשות).